

MODULE 4

TWO-DIMENSIONAL ARRAYS

A two-dimensional array is specified using two subscripts where one subscript denotes row and the other denotes column. Two dimensional array can be viewed as an array of arrays as shown below.

	Column 1	Column 2	Column 3	Column 4
Row 1	x[0][0]	x[0][1]	x[0][2]	x[0][3]
Row 2	x[1][0]	x[1][1]	x[1][2]	x[1][3]
Row 3	x[2][0]	x[2][1]	x[2][2]	x[2][3]

Declaring Two-dimensional array

The declaration statement tells the compiler the name of the array, the data type of each element in the array and the size of each dimension. A two-dimensional array can be declared as

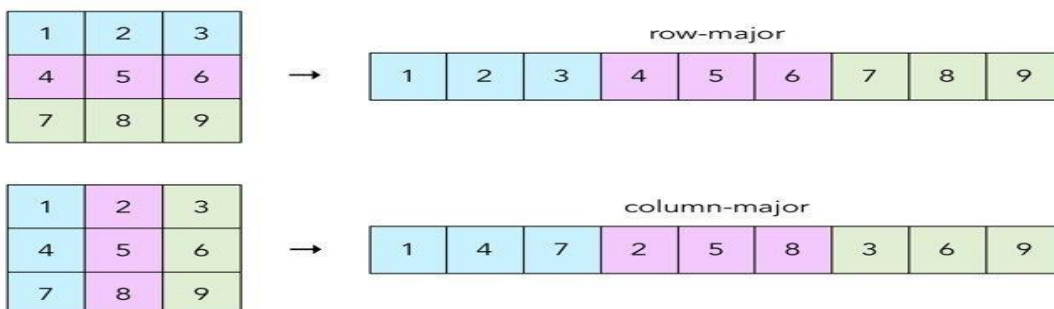
```
data_type array_name[row_size][column_size];
```

A two dimensional m X n array is an array that can contain m*n data elements and each element is accessed using two subscripts i and j where $i \leq m$ and $j \leq n$

Example: `int marks[3][5];`

A two-dimensional array called marks is declared that contains 3 rows and 5 columns.

There are two ways of storing a two-dimensional array in memory. The first way is row major order and the second is column major order. In row major order, the elements of the first row are stored before the elements of the second and third row. i.e., the elements of the array are stored row by row. In column major order, the elements of the first column are stored before the elements of second and third column i.e., the elements of the array are stored column by column.



Address of the elements of 2D array can be calculated using the formula:

$\text{Address}(A[I][J]) = B_A + w \{ M(J-1) + (I-1) \}$ if the array elements are stored in column major order.

$\text{Address}(A[I][J]) = B_A + w \{ N(I-1) + (J-1) \}$ if the array elements are stored in row major order.

Where w is the word size, M is the number of rows, N is the number of columns, I and J are the subscripts of the array element and B_A is the base address.

Example: Consider a 20 X 5 2D array Marks which has base address 1000 and the word size is 2. Now compute the address of the element Marks[18][4] assuming that the elements are stored in row major order.

$$\begin{aligned}\text{Address}(A[I][J]) &= B_A + w \{ N(I-1) + (J-1) \} \\ &= 1000 + 2 \{ 5(18-1) + (4-1) \} \\ &= 1176\end{aligned}$$

INITIALIZING TWO-DIMENSIONAL ARRAYS

A two-dimensional array can be initialized as follows:

Syntax: `data_type array_name[size]={list of values}`

Examples: `int marks[3][4]={ 1,2,3, 4, 5, 6, 7,8,9,10,11,12};`

The initialization is done row by row.

The above statement can also be written as:

```
int marks[3][4]= { {1,2,3,4} {5,6,7,8} {9,10,11,12}};
```

- While initializing the size of the first dimension can be omitted. Therefore, the below statement is valid.

```
int marks[][4]= { {1,2,3,4} {5,6,7,8} {9,10,11,12}};
```

- In order to initialize the entire 2D array to zero, we can write the following statement
`int marks[3][4]= {0};`

- If some values are missing during initialization, then it is automatically set to zero. For example, the below statement will initialize the first row and the remaining elements will be initialized to zero.

```
int marks[3][4]= { {1,2,3,4}};
```

- Individual elements can also be initialized using assignment operator as shown below:

```
marks[1][2]=10;
```

```
marks[2][1]=marks[1][1] + 10;
```

- In order to input the values from the keyboard, we must use the following code:

```
for(i=0; i<3; i++)  
for(j=0;j<4;j++)  
{  
scanf("%d", &marks[i][j]);  
}
```

ACCESSING THE ELEMENTS OF TWO-DIMENSIONAL ARRAYS

The elements of a 2D array are stored in contiguous memory locations. Two for loops are used to scan the elements. The first for loop will scan each row in the 2D array and the second for loop will scan individual columns for every row in the array.

Write a program to print the elements of a 2D array

```
#include<stdio.h>  
int main(){  
    int rows, columns;  
    int a[2][2] = {{10, 20}, {30, 70}};  
    printf("\nPrinting the 2D Array\n")  
  
for(int rows = 0; rows < 2; rows++)  
    {  
        for(int columns = 0; columns < 2; columns++)  
        {  
            printf("%d ", a[rows][columns]);  
        }  
        printf("\n");  
    }  
    return 0;  
}
```

Output:

Printing the 2D array

10 20

30 40

OPERATIONS ON TWO-DIMENSIONAL ARRAY

Two dimensional arrays can be used to implement mathematical concept of matrices. Matrix is a grid of numbers arranged in rows and columns.

Transpose: Transpose of a m X n matrix A is given as n X m matrix B where

$$B_{i,j} = A_{j,i}$$

Sum: Two matrices that are compatible can be added together storing the result in the third

matrix. Two matrices are said to be compatible when they have the same number of rows and columns.

$$C_{i,j} = A_{i,j} + B_{i,j}$$

Difference: Two matrices that are compatible can be subtracted storing the result in the third matrix. Two matrices are said to be compatible when they have the same number of rows and columns.

$$C_{i,j} = A_{i,j} - B_{i,j}$$

Product: Two matrices can be multiplied with each other if the number of columns in the first matrix is equal to the number of rows in the second matrix. A matrix of order m X n can be multiplied with the matrix of order p X q if n is equal to p.

$$C_{i,j} = \sum A_{i,k} B_{k,j} \text{ for } k=1 \text{ to } k<n$$

Write a program to transpose a matrix in c

```
#include <stdio.h>
int main()
{
    int a[10][10], transpose[10][10], r, c, i, j;
    printf("Enter rows and columns: ");
    scanf("%d %d", &r, &c);
    printf("\nEnter matrix elements:\n");
    for ( i = 0; i < r; i++)
        for ( j = 0; j < c; j++)
        {
            scanf("%d", &a[i][j]);
        }

    for ( i = 0; i < r; i++)
        for ( j = 0; j < c; j++)
        {
            transpose[j][i] = a[i][j];
        }
    printf("\nTranspose of the matrix:\n");
    for ( i = 0; i < c; i++)
    {
        for ( j = 0; j < r; j++)
        {
            printf("%d ", transpose[i][j]);
        }
        printf("\n");
    }
}
```

```
}  
    return 0;  
}
```

Output:

Enter rows and columns:

2

3

Enter matrix elements:

1	4	0	-5	2	7
---	---	---	----	---	---

Transpose of the matrix:

1	-5
---	----

4	2
---	---

0	7
---	---

Write a program to add two matrix in c

```
#include <stdio.h>
int main()
{
    int r, c, a[100][100], b[100][100], sum[100][100], i, j;
    printf("Enter the number of rows ");
    scanf("%d", &r);
    printf("Enter the number of columns ");
    scanf("%d", &c);
    printf("\nEnter elements of 1st matrix:\n");
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
        {
            scanf("%d", &a[i][j]);
        }

    printf("Enter elements of 2nd matrix:\n");
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
        {
            scanf("%d", &b[i][j]);
        }
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
        {
            sum[i][j] = a[i][j] + b[i][j];
        }
    printf("\nSum of two matrices: \n");
    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            printf("%d ", sum[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Output:

Enter the number of rows: 2

Enter the number of columns: 3

Enter elements of 1st matrix:

2 3 4 5 2 3

Enter elements of 2nd matrix:

-4 5 3 5 6 3

Sum of two matrices:

-2 8 7

10 8 6

PASSING TWO-DIMENSINAL ARRAYS TO FUNCTIONS

There are 3 ways of passing two-dimensional arrays to functions.

Passing individual elements: Individual array can be passed to the functions.

```
main()
{
    int arr[2][3] = {{1,2,3},{4,5,6}};
    func(arr[1][2]);
}

void func(int x)
{
    printf("%d",x);
}
```

Passing a Row: A row of a 2D array can be passed by indexing the array name with the row number. When we send a single row of 2D array, then the called function receives a one-dimensional array.

```
main()
{
    int arr[2][3] = {{1,2,3},{4,5,6}};
    func(arr[1]);
}

void func(int arr[])
{
```

```
    int i;
    for(i=0;i<3;i++)
        printf("%d",arr[i]);
}
```

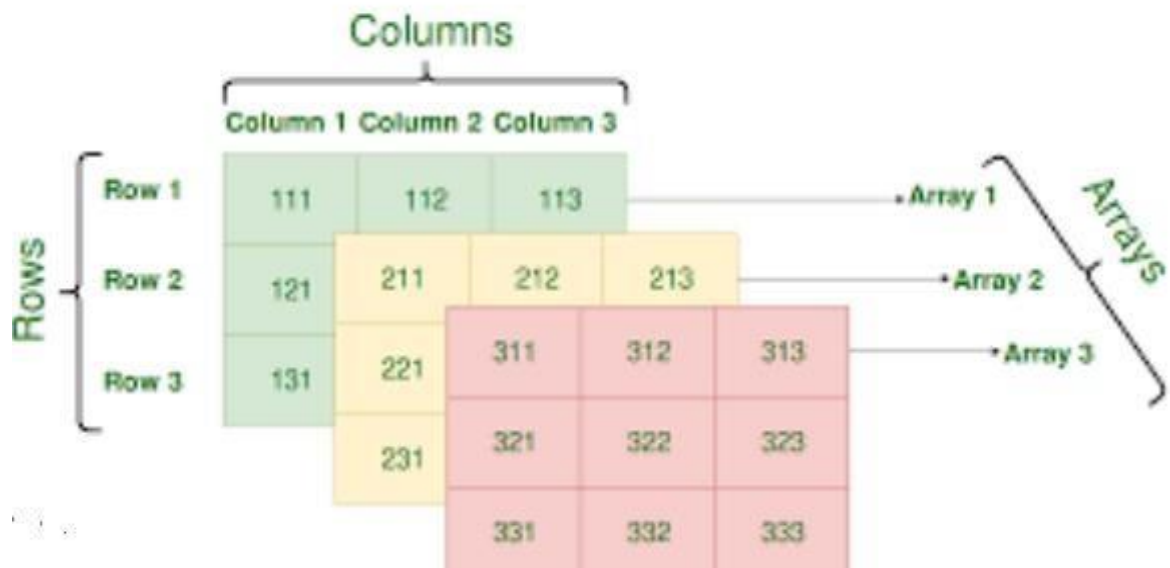
Passing an Entire 2D array: To pass a 2D array to a function, array name can be used as the actual parameter.

```
main()
{
    int arr[2][3] = {{1,2,3},{4,5,6}};
    func(arr);
}
```

```
void func(int arr[])
{
    int i,j;
    for(i=0;i<2;i++)
        for(j=0;j<3;j++)
            printf("%d",arr[i][j]);
}
```


MULTIDIMENSIONAL ARRAY

A multidimensional array is an array of arrays.



An n-dimensional array is specified using n indices. An n-dimensional $m_1 \times m_2 \times m_3 \times \dots \times m_n$ array is a collection $m_1 * m_2 * m_3 * \dots * m_n$ elements. A multidimensional array can

contain as many indices as needed and the requirement of the memory increases with number of indices.

Write a program to read and display a 2 X 2X 2 array.

```
include <stdio.h>
int main()
{
    int a[2][2][2],i,j,k;
    printf("Enter elements of the matrix: \n");
    for (i = 0; i < 2;i ++)
    {
        for (j = 0; j < 2; j++)
        {
            for ( k = 0; k < 2;k ++)
            {
                scanf("%d", &a[i][j][k]);
            }
        }
    }

    printf("\nDisplaying values:\n");
    for (i = 0; i < 2;i ++)
    { printf("\n\n");
        for (j = 0; j < 2; j++)
        { printf("\n");
            for ( k = 0; k < 2;k ++)
            {
                printf("\t arr[%d] [%d] [%d]",i,j,k,a[i][j][k]);
            }
        }
    }
    return 0;
}
```

Output:

Enter the elements of the matrix

1 2 3 4 5 6 7 8

Displaying Values:

a[0][0][0] = 1	a[0][0][1] = 2
a[0][1][0] = 3	a[0][1][1] = 4
a[1][0][0] = 5	a[1][0][1] = 6
a[1][1][0] = 7	a[1][1][1] = 8

SPARSE MATRICES

Sparse matrix is a matrix that has large number of elements with a zero value. There are two types of sparse matrices.

- Lower triangular matrix: Here, all the elements above the main diagonal have zero value
- Upper triangular matrix: In this matrix, all the elements below the main diagonal have zero value.

Lower Triangular	Upper Triangular
$\begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ -2 & -1 & 1 \end{bmatrix}$	$\begin{bmatrix} 2 & -1 & -2 \\ 0 & 4 & -1 \\ 0 & 0 & 3 \end{bmatrix}$

Tridiagonal matrix: Here, the elements are present on

- The main diagonal, it contains non-zero elements for $i=j$
- Below the main diagonal, it contains non-zero elements for $i=j+1$
- Above the main diagonal, it contains non-zero elements for $i=j-1$.

$$\begin{pmatrix} 1 & 4 & 0 & 0 \\ 3 & 4 & 1 & 0 \\ 0 & 2 & 3 & 4 \\ 0 & 0 & 1 & 3 \end{pmatrix}.$$

Array Representation of Sparse Matrices

Consider the sparse matrix given below

This sparse matrix can be represented as

3	3	2
0	1	2
1	2	4



- The first row give the number of rows(3) , number of columns (3) and number of non-zero elements(2) in the sparse matrix
- The second row specifies the location and value of first non-zero element. The first non-zero value is 2 is at (0,1) location.
- Similarly, third row specifies the location and value of the next non-zero i.e, value 4 at (1,2) location.

Write a program that shows the array representation of a sparse matrix

```
#include<stdio.h>
void sparse_matrix(int mat[3][3],int, int, int);
void main()
{
int i, j, rows, cols, num_values=0,mat[30][30];
printf("enter number of rows and columns \n");
scanf("%d %d",&rows,&cols);
printf("enter the elements of the array \n");
for(i=0;i<rows;i++)
{
for(j=0;j<cols;j++)
{
scanf("%d", &mat[i][j]);
}
}
printf("the sparse matrix is :\n");
for(i=0;i<rows;i++)
{ printf("\n");
for(j=0;j<cols;j++)
{
```

```

printf(" %d \t", mat[i][j]);
}
}
for(i=0;i<rows;i++)
{
for(j=0;j<cols;j++)
{
    if(mat[i][j] !=0)
        num_values++;
}
}
sparse_mat(mat,num_values,rows,cols);
getch();
}

```

```

void sparse_mat(int mat[30][30], int num_values, int rows, int cols)
{
int new_mat[5][3],i,j,temp_index=1;
new_mat[0][0]=rows;
new_mat[0][1]=cols;
new_mat[0][2]= num_values;
for(i=1;i<=rows;i++)
{
for(j=1;j<=cols;j++)
{
    if(mat[i-1][j-1] != 0)
    {
        new_mat[temp_index][0] = i-1;
        new_mat[temp_index][1] = j-1;
        new_mat[temp_index][2] = mat[i-1][j-1];
        temp_index ++;
    }
}
}
printf("the new matrix is:\n");
for(i=0;i<num_values;i++)
{ printf("\n");
for(j=0;j<3;j++)
{
printf(" %d \t", new_mat[i][j]);
}
}
}

```

Output:

Enter the number of rows: 2

Enter the number of columns: 2

Enter the elements of array: 1 0 0 0

The sparse matrix is:

1 0

0 0

The new matrix is:

2 2 1

0 0 1

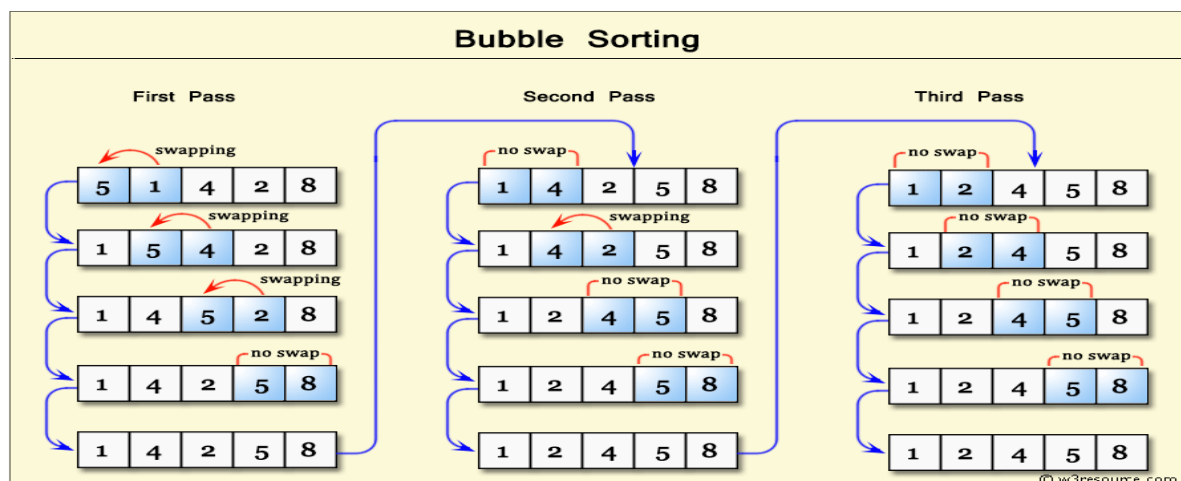
APPLICATIONS OF ARRAYS

- Arrays are widely used to implement mathematical vectors, matrices and other kinds of rectangular tables.
- Many databases include one-dimensional arrays where elements are recorded.
- Arrays are also used to implement other data structures such as strings, stacks, queues, heaps and hash tables.
- Arrays can be used for sorting elements in ascending or descending order.

CASE STUDY: SORTING

Bubble Sort

Bubble sort will start by comparing the first element of the array with the second element, if the first element is greater than the second element, it will swap both the elements, and then move on to compare the second and the third element, and so on.



```
#include<stdio.h>
void main( )
{
```

```

int i,j,n,temp,a[100];
printf("Enter the value of n\n");
scanf("%d",&n);
printf("Enter the numbers in unsorted order:\n");
for(i=0;i<n;i++)
scanf("%d", &a[i]);
for(i=0;i<n-1;i++)
{
for(j=0;j<(n-i)-1;j++)
{
if( a[j]>a[j+1])
{
temp=a[j];
a[j]=a[j+1];
a[j+1]=temp;
}
}
}
printf("The sorted array is\n");
for(i=0;i<n;i++)
{
printf("%d\n",a[i]);
}
}

```

Selection Sort

Selection sort will start by comparing the first element with other elements and finds minimum element and swaps, then it starts comparing by taking second element with other elements and so on.

```

#include<stdio.h>
int smallest(int arr[], int k, int n);
void selection_sort(int arr[] , int n);
void main( )
{
int arr[100],i,n;
printf("Enter the value of n\n");
scanf("%d",&n);
printf("Enter the numbers in unsorted order:\n");
for(i=0;i<n;i++)
scanf("%d", &a[i]);
selection_sort(arr,n);
printf(" sorted array is :\n");
}

```

```

for(i=0;i<n;i++)
printf("%d", a[i]);
getch();
}
int smallest(int arr[], int k, int n)
{
int pos=k, small=arr[k] , i;
for(i=k+1; i<n; i++)
{
if(arr[i]<small)
{
small=arr[i];
pos=i;
}
}
return pos;
}

```

```

void selection_sort(int arr[], int n)
{
int k,pos,temp;
for(k=0; k<n; k++)
{
pos=smallest(arr,k,n);
temp=arr[k];
arr[k]=arr[pos];
arr[pos]=temp;
}
}

```

Output:

Enter the number of elements in array: 6

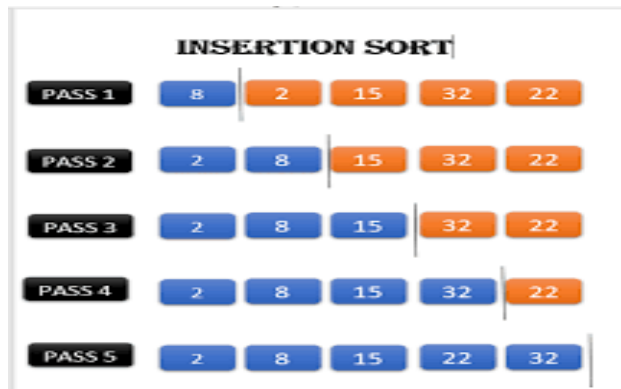
Enter the element of array: 27 72 36 63 45 54

The sorted array is

27 36 45 54 63 72

INSERTION SORT

It is a very simple sorting algorithm in which the sorted array is built one element at a time. It inserts each item into its proper place in the final list. It works by moving the current data element past the already sorted values and repeatedly interchanging it with the preceding value until it is in its correct place.



```
#include<stdio.h>
void insertion_sort(int arr[], int n);
void main( )
{
    int arr[100],I,n;
    printf("Enter the value of n\n");
    scanf("%d",&n);
    printf("Enter the numbers in unsorted order:\n");
    for(i=0;i<n;i++)
        scanf("%d", &arr[i]);
    insertion_sort(arr,n);
    printf("The sorted array is\n");
    for(i=0;i<n;i++)
    {
        printf("%d\n",arr[i]);
    }
    getch();
}

void insertion_sort(int arr[], int n)
{
    int i, j, temp;
    for(i=1;i<n;i++)
    {
        temp=arr[i];
        j=i-1;
        while((temp<arr[j]) && (j>=0))
        {
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = temp;
    }
}
```

STRINGS

A String is a null-terminated character array. Last character is array in '\0' null character.

Example: `char str[]="PROGRAM";`

Index	0	1	2	3	4	5	6	7
Characters	P	R	O	G	R	A	M	\0
Address	1000	1001	1002	1003	1004	1005	1006	1007

In the above example, `str` is a string storing the sequence of characters "PROGRAM". To store a string of length 7, we need 7+1 location (1 extra for the null character). The name of the character array is a pointer to the beginning of the string. The subscript starts with a zero. All the characters of a string array are stored in successive memory locations. In figure, we can see the address is 1000, 1001 and so on.

Declaring string variables: A string is declared like an array of characters.

Syntax: `char string_name[size];`

Ex: `char name[21];`

Size 21 means it can store up to 20 characters plus the null character. Entire storage location name is divided in to 21 boxes called bytes, each of which holds one character. Each character is element of data type `char`.

Initializing the string in the declaration:

The following code shows to initialize a string as an array of characters. We need to explicitly add the null character.

```
char str[]={ 'H','E','L','L','O','\0' };
```

We can also declare a string with size much larger than the number of elements that are initialized.

```
char str[10]="HELLO";
```

Reading Strings

Strings can be read using 3 ways

- Using `scanf()` function
- Using `gets()` function
- Using `getchar()`, `getch()` or `getche()` function

Using scanf() function

Strings can be read using scanf() by writing

```
scanf("%s",str);
```

Example:

```
char name[20];
```

```
scanf("%s", name);
```

The disadvantage of scanf() is that the function terminates as soon as it finds a blank space. For example, if the user enters Hello World, then name will contain only Hello.

Using gets() function

The string can be read by writing

```
gets(str);
```

gets() overcomes the drawbacks of scanf() function. It takes the starting address of the string which will hold the input. The string inputted using gets() is automatically terminated with a null character.

Using getchar() function

Strings can also be read by calling the getchar() function repeatedly to read a sequence of single characters and simultaneously storing it in a character array.

```
i=0;
```

```
ch=getchar();
```

```
while(ch!='*')
```

```
{
```

```
str[i]=ch;
```

```
i++;
```

```
ch=getchar();
```

```
}
```

```
str[i]='\0';
```

WRITING STRINGS

Strings can be displayed on screen using 3 ways

- Using printf() function
- Using puts() function
- Using putchar() repeatedly

Using printf() function

Strings can be displayed using printf() by writing

```
printf("%s",str);
```

%s is used to specify that we are printing string.

```
printf(“%5.3s”,str);
```

The above statement would print only the first three characters in a total field of five characters. Here, the strings are right justified. To make the string left justified, we must use a minus sign

```
printf(“%-5.3s”,str);
```

Using puts() function

The string can be displayed by writing

```
puts(str);
```

This function writes a line of output on the screen. It terminates the line with a newline character.

Using putchar()

Strings can also be written by calling the putchar() function repeatedly to print a sequence of single characters.

```
i=0;
while(str[i]!='\0')
{
    putchar(str[i]);
    i++;
}
```

Summary of functions used to read and write characters

Function	Operation
getchar()	Used to read a character from the keyboard, waits for carriage return(enter key). It can be used to input any key including RETURN, TAB and ESC
getch()	Is an alternative for getchar(). It waits for a keypress, after which it return immediately.
getche()	Similar to getch(). The only difference is that it echoes the character on screen
putchar()	Used to write a character to the screen. Returns the character written or EOF (-1) if an error occurs.

Write a program to print the following pattern

```
H
H E
H E L
H E L L
H E L L O
```

```
H E L L O
H E L L
H E L
H E
H
```

```
#include<stdio.h>
void main()
{
int i,w,p;
char str[]="HELLO";
printf("\n");
for(i=0;i<5;i++)
{
p=i+1;
printf("\n % -5.*s",p,str);
}
printf("\n");
for(i=4;i>=0;i--)
{
p=i+1;
printf("\n % -5.*s",p,str);
}
getch();
}
```

sprintf() Function

It is similar to printf() function. The only difference is that the formatted output is written to a memory area rather than directly to standard output(screen). It is useful in situations when formatted strings in memory have to be transmitted over a communication channel or to a special device. Syntax is as follows:

```
int sprintf( char * buffer, const char * format [ , argument , . . . ] );
```

Where

Buffer is the place where string needs to be stored.

arguments command is an ellipsis so you can put as many types of arguments as you want

format is the string that contains the text to be printed

```
#include<stdio.h>
void main()
{
char buf[100]; int
num=10;
```

```
sprintf(buf, "num=%3d", num);  
}
```

SUPPRESSING INPUT

The scanf() function can be used to read a field without assigning it to any variable. This is done by preceeding that field's format code with a *. For example,

```
scanf("%d *c %d", &hr, &min);
```

The time is read as 9:05. Here colon would be read but not assigned to anything.

Using a scanset

Scanset is used to define a set of characters which may be read and assigned to the corresponding string. It is defined by placing the characters inside square brackets prefixed with a % as shown in the example.

```
%["aeiou"]
```

When we use the above scanset, scanf() will continue to read characters and put them into the string until it encounters a character that is not specie in the scanset.

```
#include<stdio.h>  
void main()  
{  
char str[10];  
scanf("%[aeiou]", str);  
printf(" string is : %s", str);  
}
```

The following code will accept characters other than those specified in the scanset.

```
scanf("%[^aeiou]", str);
```

The following code will accept any character enclosed in opening and closing square brackets.

```
scanf("%[0123456789. ^ [] () _ + -$%&*]", str);
```

The following code will accept any character from small a to z. Range is specified using a hyphen.

```
scanf("%[a-z]",str);
```

scanf() Function

It accepts a string from which to read input. It is similar to scanf() function except that the first argument of scanf specifies a string from which to read, whereas scanf can only read from standard input. Syntax is as follows:

```
intscanf(const char * str, const char * format, [p1,p2 , . . . ]);
```

Here, scanf() reads data from str and stores then according to the parameters format into the locations given by additional arguments.

```
scanf(str, "%d", &num);
```

Here, scanf() takes 3 arguments. The first is str that contains data to be converted. The second is string containing a format specifier that determines how the string is converted. The third is a memory location to place the result of the conversion.